

DIO: Hybrid Cost Routing, Session Affinity, and Sound Admission for Multi-Instance LLM Serving over Stock vLLM

Keyush Nisar¹, Krishil Parikh¹, Krisha Maisheri¹,
Aruna Gawade¹, Nilesh T. Rathod¹, Angelin Florence A¹

¹Department of Computer Engineering, Dwarkadas J. Sanghvi College of Engineering, Mumbai, India.

Contributing authors: nisarkeyush3@gmail.com;
krishilparikh75@gmail.com; krishamaisheri16@gmail.com;
aruna.gawade@djsce.ac.in; nilesh.rathod@djsce.ac.in;
angelin.florence@djsce.ac.in;

Abstract

Operators often run several stock vLLM instances behind Round-Robin or least-connections balancers. Those balancers treat every peer as equal, yet queues, KV-cache pressure, and temporary slowdowns still diverge across replicas. Latency models learned only from end-to-end telemetry also make a poor basis for hard admission. We present **DIO** (**dio-serve**), a thin OpenAI-compatible gateway that ranks backends with dual-timescale NLMS, adjusts scores using Prometheus `/metrics` exported by each vLLM instance, keeps multi-turn sessions on the replica that already holds the shared prefix, and separates admission from absolute predicted latency. The design requires no engine patches.

On dual Tesla T4 GPUs with Qwen2.5-3B and stock vLLM, DIO reduces p99 latency under controlled service-time asymmetry relative to Round-Robin and $d=2$ recursive least squares, improves multi-turn tails when session affinity keeps turns on the warm replica, maintains high session stickiness under concurrent load, and avoids the over-rejection that absolute- $\hat{y}$ admission produces under a tight SLO. When backends are matched, ranking remains near Round-Robin parity. We also report a negative result we consider the most transferable finding here: the same online model that ranks replicas well is too poorly calibrated in absolute terms to gate an SLO, so the natural “reject if $\min \hat{y} > \text{SLO}$ ” policy rejects $\sim 40\%$ of a load that rank-only and empirical gating complete in full. Source code is publicly available.

Keywords: LLM serving, multi-instance load balancing, vLLM, hybrid routing, prefix-cache affinity, admission control, NLMS, Prometheus metrics.

1 Introduction

1.1 Motivation

Practitioners commonly run *multiple* stock vLLM [1] (or SGLang/TGI [2–4]) instances and put Nginx [5], Kubernetes Services [6], Envoy [7], or Ray Serve [8] in front with Round-Robin or least-connections. Single-node engines optimize continuous batching [9, 10]; they do not solve *which backend* should take the next request. Classic L7 balancers [5, 11] treat backends as equal capacity units.

In production, effective cost still diverges even on identical GPU SKUs: pending queues grow unevenly, KV-cache utilization spikes on one replica, prefix caches favor sessions pinned to a peer, and host interference can temporarily slow one process. The result is avoidable tail latency [12]. Co-designed cluster schedulers [13–15] can react with deep engine hooks or live KV migration, but many operators need a non-invasive front door that wraps stock OpenAI-compatible [16] engines without patches.

A second problem is quieter but equally important: online latency models learned from black-box e2e telemetry are often accurate only as *relative rankers*. On dual Tesla T4 we measure dual-timescale NLMS [17] MAPE of $\sim 90\text{--}130\%$. Systems that still gate admission on “reject if $\min \hat{y} > \text{SLO}$ ” inherit that error and can starve healthy traffic.

1.2 Approach

We design **DIO** (**dio-serve**) as a thin control plane for the multi-instance pattern (Figure 1). The closed loop uses coarse request telemetry (e2e latency, tokens, gateway pending) *and*, when available, stock engine metrics already exported on each vLLM Prometheus [18] endpoint—still HTTP-only, still zero source modification. Three mechanisms work together:

- (A) **Hybrid cost routing.** NLMS ranks workers online; scraped KV usage, waiting requests, and prefix hit rate correct that ranking without engine hooks (Section 4).
- (B) **Session/prefix affinity.** Multi-turn and agent clients [19, 20] benefit when follow-ups land where a shared prefix is already cached [21–23]. DIO scores stickiness and reports affinity hit rate (Section 5).
- (C) **Sound admission.** Ranking uses $\arg \min S_w$; admission is a separate policy (**rank_only**, **empirical**, or diagnostic **absolute**) so absolute $\hat{y}$ cannot thrash reject/admit decisions [24, 25] (Section 6).

1.3 Results preview

On multi-seed dual-T4 runs, homogeneous peers keep NLMS near Round-Robin parity (Section 7.3). Under controlled service-time asymmetry, NLMS improves p99 relative to Round-Robin and $d=2$ RLS (Section 7.5). On multi-turn workloads, session affinity keeps sessions on the warm replica and drives the p99 improvement, while the scraped engine-metric terms contribute only a secondary signal — an attribution we establish by decomposition in Section 7.7 rather than assume; affinity holds under concurrent load (Section 7.8), and empirical or rank-only admission preserve completions where absolute predicted-latency gating rejects many requests (Section 7.9).

1.4 Contributions

1. **Session/prefix affinity for multi-turn routing.** Prefix-hash stickiness plus an optional engine prefix-hit bonus; on dual-T4 multi-turn workloads stickiness reaches 1.00 vs. RR 0.50 at sequential load and 0.998 vs. 0.75 under concurrent load, driving 40.9% p99 improvement.
2. **Hybrid cost routing (complementary signal).** Joint cost fusing online NLMS with scraped vLLM /metrics; the scraped terms contribute a small but measurable signal when concurrency is real, confirmed by a 2×2 factorial mechanism study. Under the dual-T4 campaign of this paper ($n=10$), the dominant term in the hybrid arm is the cache bonus of Contribution 1, not the engine-gauge terms — a qualification we audit transparently in Section 7.7.
3. **Sound admission under weak absolute prediction.** Rank-only / empirical / absolute modes; empirical and rank-only complete all requests under a tight SLO while absolute over-rejects ($n=10$).
4. **Open gateway and multi-seed evaluation.** Open-source `dio-serve` on dual-T4 + Qwen2.5-3B [26], with honest homogeneous baselines and reproducible skew, multi-turn, and admission suites.

2 Background and related work

2.1 Single-node engines and prefix/KV systems

vLLM [1] popularized PagedAttention and continuous batching; alternatives revisit memory management without paging [27]. SGLang [2, 4] improves structured decoding and RadixAttention prefix reuse. TGI [3] targets production APIs. Hybrid iteration policies include Sarathi-Serve [10] and FastServe [28]. Disaggregated prefill/decode systems [25, 29–31] optimize pipeline stages and KV placement. Prefix/KV reuse [21–23, 32, 33] raises the value of *where* a request lands. These systems excel on one node or a co-designed cluster; multi-worker routing in front of *stock* engines remains external.

2.2 Cluster orchestrators and classical serving

Ray Serve [8] and Triton [34] emphasize actors and model repositories with coarse metrics. Earlier DNN serving stacks (Clipper [35], INFaaS [36], Clockwork [24]) established predictability and model-selection themes that LLM gateways inherit. Orca [9] advances iteration-level batching. AlpaServe [37] and CoCoServe [38] focus on placement and multiplexing. Mélange [39] targets heterogeneous capacity tables. NexusSched [13] combines predictive routing with engine co-design and multi-feature predictors ($O(d^2)$ updates). Llumnix [14] migrates live KV state. LoongServe [15] targets long-context elastic parallelism. ServerlessLLM [40] optimizes cold-start loading.

Inference-aware gateways: what already ships. We state the competitive baseline plainly, because metrics-aware and prefix-aware routing over unmodified vLLM is no longer a gap in the literature — it is deployed infrastructure. The Kubernetes Gateway API Inference Extension [41] and its Endpoint Picker select replicas using scraped vLLM queue depth, KV utilization, and LoRA state; llm-d [42] and Google’s GKE Inference Gateway [43] build on it. NVIDIA Dynamo [44] maintains a radix tree of cached prefixes and scores endpoints by prefill/decode block overlap. The vLLM production-stack router [45] ships round-robin, session, prefix-aware, and

KV-aware policies. Alibaba’s ACK/ACS gateway [46] offers estimated and precise prefix-cache-aware modes, and the SGLang router [47] does cache-aware balancing. Envoy AI Gateway [48] adds LLM-specific routing to Envoy. Any of these subsumes the individual mechanisms of contributions 1–2.

What remains distinct in DIO. We therefore do not claim novelty for “gateway scrapes engine metrics” or “gateway pins sessions to warm replicas.” Three narrower elements are, to our knowledge, not combined in the systems above. First, the routing signal is a *dual-timescale NLMS* per-worker latency model (Section 3.2) with $O(1)$ updates from black-box e2e telemetry only: the deployed gateways above rank on instantaneous scraped gauges rather than a learned per-replica service-rate estimate, and the closest learned comparators either need engine co-design or $O(d^2)$ updates [13]. Second, we *decouple admission from ranking* and show why this matters: the same online model that ranks well is badly miscalibrated in absolute terms (MAPE $\sim 90\text{--}130\%$, Section 7.3), so an SLO gate of the form “reject if $\min \hat{y} > \text{SLO}$ ” — the natural design — rejects $\sim 40\%$ of a load that empirical and rank-only gating complete in full (Section 7.9). We regard this negative calibration result, not the routing gain, as the paper’s most transferable finding: it is a caution that applies to any predictive admission controller built on black-box telemetry. Third, we report a 2×2 factorial that separates the engine-metric contribution from the affinity contribution (Section 7.7); the gateway systems above publish aggregate wins without this decomposition, and our own audit shows how easily the two are conflated. Section 8 compares published numbers and test regimes against DistServe, Mooncake, MemServe, Llumnix, NexusSched, and classical L7 balancers.

2.3 Adaptive prediction, queues, and admission

NLMS stability and step-size practice follow adaptive filter theory [17, 49]; recursive least squares is the natural $O(d^2)$ comparator. Queue wait uses a batch-amortized Little-style heuristic [50, 51]. SJF-style ranking motivates predicted service routing [52]. Roofline [53] inspires soft memory ceilings recast as routing penalties. Goodput- and SLO-aware LLM serving [25, 54, 55] motivates treating admission as a separate policy from ranking. Simulation frameworks such as Vidur [56] characterize offline capacity; DIO targets online multi-instance ranking under weak absolute models.

3 System design

3.1 Architecture and software artifact

Figure 1 shows `dio-serve`, the gateway used in all experiments below. Clients issue OpenAI-compatible HTTP requests. A FastAPI control plane ranks backends with dual-timescale NLMS, optionally folds in scraped engine metrics, applies session affinity and admission, then proxies to a stock vLLM worker. Completions update per-worker models from observed e2e latency and token counts. A Go/gRPC research path exists in the repository but is not used for the results in this paper.

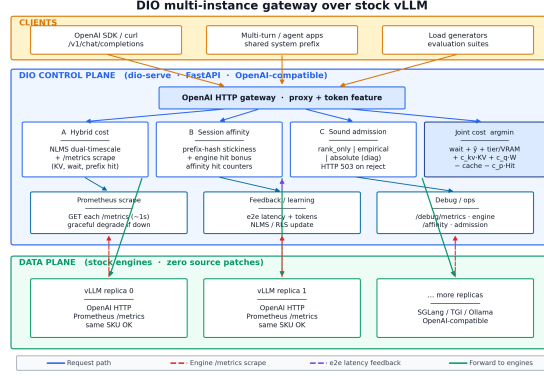


Fig. 1 DIO multi-instance gateway. Clients call `dio-serve`; the control plane ranks stock vLLM replicas with NLMS, optional Prometheus scrape, session affinity, and admission (no engine patches).

Telemetry uses four signals: gateway `e2e` latency; token counts from response `usage` or a tokenizer estimate; per-backend pending depth; and optional HTTP scrape of each engine `/metrics`. Soft and hard VRAM limits come from configuration or KV usage. We do not sample NVML SM/power/temperature and do not modify engine source [13, 14].

A typical launch is `dio serve -b e0=URL0 -b e1=URL1`. Background tasks scrape `/metrics` every 1s by default and probe health. Debug endpoints report metrics, engine state, affinity, and admission counters. Scheduling overhead is about 10–15 μ s per request (Section 7.2). Source code, library multi-seed harnesses, and dual-T4 scripts are public (Declarations).

3.2 Dual-timescale NLMS latency model

For worker w and token count N we use the linear model

$$\hat{y}_w = s_w N + b_w, \quad (1)$$

with slope s_w (ms/token) and intercept b_w (ms). After a completion with residual $e = y - \hat{y}$, NLMS updates

$$s \leftarrow s + \mu \frac{e}{N}, \quad b \leftarrow b + \mu_b e, \quad (2)$$

using dual rates $\mu_{\text{fast}}=0.1$, $\mu_{\text{slow}}=0.01$, $\mu_b=0.005$, and

$$s^{\text{eff}} = \alpha s^{\text{fast}} + (1 - \alpha) s^{\text{slow}}, \quad \alpha=0.8. \quad (3)$$

Priors are $s_0=2.0$ ms/token and $b_0=150$ ms. Arithmetic per update is $O(1)$; slopes typically leave the prior after about 15–20 completions per worker. Figure 2 shows a simulated slope jump. The token feature N prefers Hugging Face tokenizer counts plus planned `max_tokens`, with fallback `len(prompt)/4 + max_tokens`.

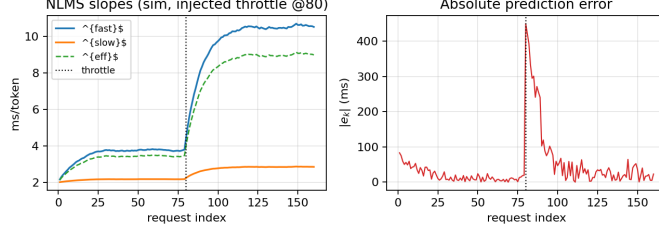


Fig. 2 Simulated dual-timescale NLMS under an injected slope jump at request 80. s^{fast} adapts first; s^{slow} lags. Not dual-T4 wall-clock traces.

3.3 Baseline joint cost

Absent hybrid metrics, each healthy worker scores

$$S_w^{\text{base}} = \text{wait}_w + \hat{t}_w + \text{tierCost}_{r,w} + \text{vramCost}_w - \text{cacheBonus}_w, \quad (4)$$

with $\hat{t}_w = s_w^{\text{eff}} N + b_w$, $\text{wait}_w = (Q_w/B)\bar{t}_w$ ($B=8$), soft VRAM penalty when free VRAM < 4 GB, hard exclusion when free VRAM < 2.4 GB and $N > 1000$, and tier mismatch soft cost 500 ms (hard block for large-on-small). Route to $\arg \min_w S_w$ in $O(|W|)$ time. Strategies also include Round-Robin, Least-Loaded, static slopes, and $d=2$ RLS as head-to-head baselines under the *same* cost shell when applicable.

4 Hybrid cost routing with engine metrics

4.1 Problem: black-box ranking misses engine ground truth

NLMS observes only gateway e2e latency and tokens. That signal is delayed, noisy, and confounded by batching. Meanwhile stock vLLM already exports internal state via Prometheus [18]: GPU KV-cache utilization, number of waiting and running requests, and prefix-cache hit counters. Prior multi-instance balancers either ignore these signals or require invasive co-design [13, 14]. DIO’s hybrid design stays coarse: **scrape what the engine already publishes**, fold it into the same millisecond-scale cost used for ranking, and keep zero source modification.

4.2 Metric scrape and snapshot

For each backend i with base URL U_i and metrics path (default `/metrics`), a background task issues `GET U_i/metrics` and parses Prometheus exposition text. Metric name variants across vLLM versions are handled (e.g. `vllm:gpu_cache_usage_perc` vs. `vllm:kv_cache_usage_perc`; prefix hit rate from a direct gauge or from hits/queries counters). Each scrape yields a snapshot

$$\mathcal{E}_w = (\text{KV}_w, W_w, R_w, \text{Hit}_w, \text{ok}_w), \quad (5)$$

where $\text{KV}_w \in [0, 1]$, W_w and R_w are waiting/running request counts, $\text{Hit}_w \in [0, 1]$ is prefix-cache hit rate, and ok_w is false on HTTP/parse failure. On failure, hybrid terms contribute zero (graceful degrade to NLMS-only ranking). Scrape interval defaults to 1 s—fast enough to track KV pressure, slow enough not to contend with decode.

4.3 Hybrid joint cost

When ok_w is true, the full score is

$$S_w = \text{wait}_w + \hat{t}_w + \text{tierCost}_{r,w} + \text{vramCost}_w + c_{kv} \text{KV}_w + c_q W_w - \text{cacheBonus}_w - c_p \text{Hit}_w. \quad (6)$$

Default hybrid coefficients (same ms units as other terms): $c_{kv}=800$, $c_q=50$ per waiting request, $c_p=150$. These were chosen so that a near-full KV cache ($\text{KV} \approx 1$) contributes on the order of one long-request decode budget, a moderate waiting queue ($W \approx 10$) is comparable to a soft VRAM penalty, and a high engine prefix hit rate is of similar scale to the session affinity bonus $C=200$ ms—heuristic parity with existing base cost terms. Base tier/cache/VRAM coefficients in earlier library microbenches showed $\pm 50\%$ relative insensitivity. For the hybrid triple we report a controlled library $\pm 50\%$ sensitivity check (Section 9); a full dual-T4 grid search of (c_{kv}, c_q, c_p) remains open. Operators may override all three via `DIO_*` configuration. The dual-T4 multi-turn result (Section 7.7) uses the defaults above.

If $\text{KV}_w > 0.95$ and $N > 500$, worker w is hard-blocked for that request (engine-reported near-full KV). Free-VRAM estimates may be updated as $\text{free}_w \approx \text{total}_w(1 - \text{KV}_w)$ when totals are known. Hybrid cost is not a claim of SM utilization, power, or temperature awareness, and it is not a substitute for NLMS: when backends differ only in learned service slope and metrics look similar, NLMS ranking still dominates. The contribution is the combination of a lightweight online ranker with non-invasive engine ground truth.

Algorithm 1 Worker selection with hybrid scrape, affinity, and admission

Require: request r , workers W , mode m , optional snapshots $\{\mathcal{E}_w\}$

- 1: *Background (async)*: periodically scrape each backend `/metrics` into $\mathcal{E}_w$ (KV, W , Hit); on failure leave hybrid terms at 0
 - 2: $N \leftarrow \text{TokenFeature}(r)$; $w^* \leftarrow \perp$; $S_{\min} \leftarrow \infty$
 - 3: **for all** $w \in W$ healthy and tier-feasible **do**
 - 4: $\hat{t}_w \leftarrow \text{Predict}(w, N)$
 - 5: **if** VRAM/KV hard-blocked($w, N, \mathcal{E}_w$) **then continue**
 - 6: **end if**
 - 7: $S_w \leftarrow \text{wait} + \hat{t}_w + \text{tier} + \text{vram} + c_{kv} \text{KV}_w + c_q W_w - \text{cacheBonus}(r, w) - c_p \text{Hit}_w$
 - 8: **if** $S_w < S_{\min}$ **then** $S_{\min} \leftarrow S_w$; $w^* \leftarrow w$
 - 9: **end if**
 - 10: **end for**
 - 11: **if** $w^* = \perp$ **or** AdmissionReject($S_{\min}, m$) **then**
 - 12: **return** HTTP 503 with Retry-After
 - 13: **end if**
 - 14: RecordPrefixAffinity(r, w^*); **return** w^*
-

5 Session and prefix-cache affinity

5.1 Problem: multi-turn locality is underserved by RR

Round-Robin and least-connections scatter multi-turn sessions across replicas. Each turn that lands on a cold replica re-prefills shared system prompts and agent context, missing in-engine prefix caches [4, 21, 22]. Cache-aware and prompt-scheduling systems [23, 32] show large gains when locality is preserved, but they often assume co-designed clusters. DIO targets the common case: several stock replicas behind one OpenAI gateway, multi-turn or agentic clients [19, 20], no engine patches.

5.2 Affinity mechanism

On each admitted request with prompt text p , DIO computes a stable prefix hash over the first 256 characters:

$$h(p) = \text{hash}(p[0:256]) \bmod 2^{32}. \quad (7)$$

A map $M : h \mapsto w$ records the last worker that successfully served that prefix. When scoring worker w , if $M[h(p)] = w$, a cache affinity bonus $\text{cacheBonus}_w = C$ (default $C=200$ ms; raised in multi-turn experiments) is subtracted from S_w . Independently, the hybrid term $c_p\text{Hit}_w$ prefers workers the *engine* reports as currently caching well. After admission, DIO sets $M[h(p)] \leftarrow w^*$.

DIO maintains `affinity_decisions` and `affinity_hits` (true when the chosen worker already owned the prefix). Exposed hit rate `affinity_hits/affinity_decisions` is the primary stickiness metric for multi-turn evaluation; operators can also compare engine-reported prefix hit rates under DIO vs. RR when live metrics are available. Affinity is request-level stickiness complementary to RadixAttention [4]: DIO increases the chance that the engine’s own cache is warm; it does not implement a second cache. The `no_cache` ablation disables the bonus for sensitivity studies.

6 Sound admission under unreliable latency prediction

6.1 Problem: absolute gates trust a weak model

Many predictive schedulers implicitly assume $\hat{y}$ is calibrated enough to compare against an SLO in absolute milliseconds. Section 7.3 shows that assumption fails for NLMS on dual-T4 (MAPE $\sim 90\text{--}130\%$). Cold-start priors and batching can make S_w larger than any reasonable SLO even when true latency is fine. Gating on $\min_w S_w > \text{SLO}$ then rejects healthy traffic—or, if priors are optimistic, admits work that will miss the SLO. Predictability-first serving [24] and goodput-oriented LLM systems [25, 54] motivate treating admission as a policy that must not inherit model bias.

6.2 Three admission modes

Ranking is always $\arg \min_w S_w$ among feasible workers. Admission is separate:

- **rank_only**: never reject for SLO based on S_w ; only hard VRAM/tier/KV blocks apply. NLMS is used only for ranking.
- **empirical** (default): after warm-up (≥ 8 samples in a window of length 64), reject only if the empirical p -th percentile (default $p=95$) exceeds the SLO *and* the best

score lies in the worst quartile of feasible workers. Cold-start does not reject on $\hat{y}$ alone.

- **absolute** (diagnostic): reject if $\min_w S_w > \text{SLO}$. Sensitive to MAPE and cold-start scale; kept to measure disagreement with the active mode.

Algorithm 2 AdmissionReject($S_{\min}, m$)

Require: best score $S_{\min}$, mode m , SLO L , recent e2e window $\mathcal{Y}$

```

1: if  $m = \text{rank\_only}$  then return false
2: end if
3: if  $m = \text{empirical}$  then
4:   if  $|\mathcal{Y}| < 8$  then return false           ▷ warm-up: do not trust  $\hat{y}$ 
5:   end if
6:    $q \leftarrow \text{Percentile}(\mathcal{Y}, p)$ 
7:    $Q_{75} \leftarrow \text{Percentile of current feasible scores at 75\%}$ 
8:   return  $(q > L) \wedge (S_{\min} \geq Q_{75})$ 
9: end if
10: return  $(S_{\min} > L)$                       ▷ absolute

```

Counters `would_reject_absolute`, `would_admit_absolute`, and `absolute_vs_active_disagree` record, on every pick, whether absolute mode would have made a different decision; these are the primary metrics for admission safety (Section 7.9). Rejected requests return HTTP 503 with `Retry-After`.

7 Experimental evaluation

7.1 Methodology

Hardware and software. Dual NVIDIA Tesla T4, stock vLLM [1], Qwen2.5-3B-Instruct [26], Hugging Face tokenizer, and the `dio-serve` gateway. Homogeneous and controlled-skew suites use `run_workshop_final_suite.py`; multi-turn hybrid, affinity, and admission suites use `run_gpu_abc_suite.py`. Library multi-seed harnesses (`run_abc_experiments.py`) complement GPU cells with coefficient and ablation checks.

Regimes (non-interchangeable).

1. **Regime A — homogeneous dual-T4 (real):** identical peers, `max_tokens=32`, $n=10$ seeds $\times$ 30 requests.
2. **Regime B — legacy Locust pilot (secondary):** longer-decode ShareGPT [57]/arXiv/Azure-style traces, single seed; absolute seconds not mixed with Regime A ms.
3. **Regime C — service-time asymmetry:** (i) CPU multi-seed simulation $n=10$; (ii) real dual-T4 with one peer under a controlled $\times 2$ end-to-end service-time inflation, $n=10$.
4. **Regime D — physically heterogeneous GPUs (real):** two *different* GPU SKUs (NVIDIA L4 24 GB and A100 40 GB) serving identical weights under stock vLLM, no injected delay, $n=10$ seeds. Where Regime C emulates a slow peer, Regime D measures one (Section 7.6).

5. **Real multi-turn hybrid / affinity / admission (primary for mechanisms A–C):** dual-T4 + stock vLLM, multi-turn long system prefixes, live `/metrics` scrape, $n=10$ seeds, including affinity under concurrent load (Sections 7.7–7.9).

Baselines and metrics. Round-Robin (RR), Least-Loaded (LL), and RLS [17] ($d=2$, $\lambda=0.99$) under the same engines where applicable. Primary metrics: p50/p99 e2e mean $\pm$ std; MAPE; fraction to backend e0; affinity hit rate / stickiness; admission reject rate, completions, and absolute-vs-active disagreement. When the routing strategy is RR or LL, DIO still runs a *shadow* dual-timescale NLMS predictor on every completion for diagnostics; the MAPE column is that shadow model, not evidence that RR routes by $\hat{y}$. We do not claim a matched multi-node Orca [9] or vLLM-distributed bake-off.

7.2 Control-plane overhead

On a scalability microbench with many concurrent mock workers, instrumented scheduling completes in approximately $14\mu\text{s}$ per request (worker scan, cost evaluation, prefix hash). Per-worker state is $\sim 256\text{ B}$. The control plane is not the bottleneck relative to LLM decode (Figure 3).

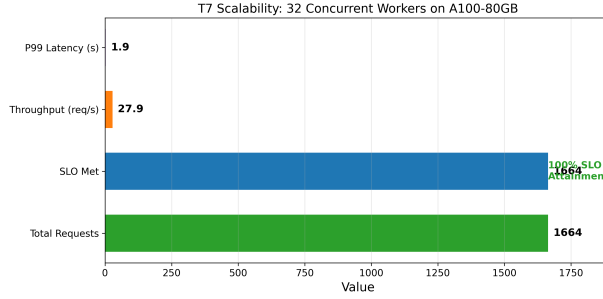


Fig. 3 Control-plane microbench. Scheduling overhead remains $\sim 14\mu\text{s}$; plotted e2e latency is synthetic worker delay.

7.3 Regime A: homogeneous dual-T4 and MAPE

Table 1 is the primary homogeneous multi-seed matrix. On *identical* backends, NLMS stays near Round-Robin on mean p99 ($-2.6\% \pm 1.0\%$)—the expected outcome when learned service costs match. Secondary signals favor stability: NLMS keeps tight p99 std (31 ms) while Least-Loaded collapses to one worker (100% to e0, p99 std 430 ms); RLS mis-routes (frac e0 0.12).

NLMS MAPE on dual-T4 is high ($123 \pm 6\%$ with tokenizer). HF tokenizer vs. byte heuristic (W2, $n=3$): MAPE $117.5 \pm 10.9\%$ vs. $89.7 \pm 4.9\%$ —tokenizer is worse, so error is structural (linear model, cold start, batching), not mere token counting. Routing uses $\arg \min S_w$; ranking under asymmetry is the success criterion (Regime C). Absolute admission that trusts $\hat{y}$ is therefore unsafe (Section 7.9).

7.4 Regime B: legacy Locust pilot (secondary)

Table 2 retains a single-seed Locust pilot (longer decode, four logical workers). Absolute seconds are **not** comparable to Regime A milliseconds; we report it only for continuity with earlier pilots.

Table 1 Regime A (real, primary). Dual-T4 + vLLM + Qwen2.5-3B, HF tokenizer, $n=10 \times 30$, `max_tokens=32`. Mean $\pm$ std.

Strategy	p99 (ms)	MAPE (%)	frac e0	vs RR
NLMS	3413 $\pm$ 31	123 $\pm$ 6	0.45 $\pm$ 0.04	-2.6 $\pm$ 1.0%
RLS ($d=2$)	3466 $\pm$ 11	132 $\pm$ 6	0.12 $\pm$ 0.02	-4.2 $\pm$ 0.6%
Round-Robin	3326 $\pm$ 17	88 $\pm$ 2	0.50 $\pm$ 0.00	—
Least-Loaded	3313 $\pm$ 430	141 $\pm$ 13	1.00 $\pm$ 0.00	+0.4 $\pm$ 13%

Table 2 Regime B (legacy single-seed Locust pilot). Not multi-seed; different setup from Table 1.

Trace	Strat.	p99 (s)	RPS	Fail %
ShareGPT	NLMS	67	0.37	0
ShareGPT	RR	81	0.29	0
arXiv	NLMS	52	0.36	0
arXiv	RR	51	0.30	0
Azure	NLMS	48	0.33	0
Azure	RR	47	0.36	0

7.5 Regime C: service-time asymmetry

We evaluate ranking under a clear service gap in simulation and on real dual-T4 hardware. In simulation ($n=10$), with distinct true (b, s) pairs, NLMS places 97.5% of traffic on the fast worker and cuts p99 by $38.3\% \pm 2.0\%$ vs. RR (Table 3, left). RLS fails to rebalance (frac fast 0.40 ± 0.18). Local `pick()` overhead is NLMS $\approx 9.6 \mu s$ and RLS $\approx 9.4 \mu s$ at $d=2$.

On real dual-T4 ($n=10$), we apply a controlled $\times 2$ end-to-end service-time inflation on peer e1 via an instrumented latency path (`latency_delay_proxy`) while both peers still run real decode on real T4 GPUs. This creates a reproducible slow-peer condition that models temporary slowdowns (interference, uneven queueing, thermal pressure) without a second GPU SKU. Table 4 (right): NLMS p99 3427 ± 38 ms vs. RR 6632 ± 37 ms ($48.3\% \pm 0.7\%$ improvement). RLS is weaker and noisier ($23.2\% \pm 22.9\%$). Mean frac to e0 under NLMS is 0.47 ± 0.08 . NLMS p99 stays close to the homogeneous Regime A level because the router steers work away from the slower peer; Round-Robin pays the full $\times 2$ penalty on half the traffic. Homogeneous longer decode (`max_tokens=128`, $n=3 \times 20$) remains tied with RR (13835 ± 73 vs. 13821 ± 88 ms).

Table 3 Regime C simulation. Slope-skew dual workers, $n=10 \times 200$.

Metric	NLMS	RR
Frac. to fast	0.975 $\pm$ 0.00	0.50 $\pm$ 0.00
p99 (sim)	1411 $\pm$ 34	2287 $\pm$ 49
p99 vs RR	38.3% $\pm$ 2.0%	

Table 4 Regime C real dual-T4, controlled $\times 2$ asymmetry, $n=10 \times 30$. NLMS p99 near Regime A reflects avoidance of the slower peer.

Strat.	p99 (ms)	frac e0	vs RR
NLMS	3427 $\pm$ 38	0.47 $\pm$ 0.08	48.3 $\pm$ 0.7%
RLS	5087 $\pm$ 1505	0.37 $\pm$ 0.19	23.2 $\pm$ 22.9%
RR	6632 $\pm$ 37	0.50 $\pm$ 0.00	—

7.6 Regime D: physically heterogeneous GPUs

Regime C establishes that DIO ranks correctly when a peer is *made* slow. The natural objection is that an injected $\times 2$ inflation is a proxy the scheduler may find easier than real hardware, where the gap is neither constant nor uniform across the request mix. Regime D removes the proxy: two different GPU SKUs, no instrumented delay path, identical weights, stock vLLM on both. The question is narrower than M elange’s [39]: that work asks which mix of GPU types to *provision* against a cost objective, whereas we ask whether a gateway handed a heterogeneous fleet learns the difference online, from black-box telemetry alone, with no capacity table and no profiling pass.

Setup. One NVIDIA L4 (24 GB, GDDR6, ≈ 300 GB/s) and one A100 (40 GB, HBM2e, ≈ 1555 GB/s), each running an unmodified `vllm.entrypoints.openai.api_server` with Qwen2.5-3B-Instruct, `--gpu-memory-utilization 0.85` on both, in a single region. The DIO gateway runs on a separate CPU node so that neither worker enjoys a loopback advantage. The $\approx 5\times$ memory-bandwidth ratio is the relevant asymmetry: autoregressive decode is bandwidth-bound, so it is precisely the per-token slope s in $\hat{y} = sN + b$ that the two SKUs should differ on.

Declared capacity. Each backend’s VRAM is declared explicitly (`--vram`), unlike the Regime C harness where both peers inherited the 24 GB default. On identical T4s that default was harmless; across SKUs it would mis-state A100 capacity and bias the `vram` term of the joint cost, so Regime D plumbs it through.

Workload design: slope identifiability. Regimes A–C fix `max_tokens`, which leaves the token feature N nearly constant. Under a near-constant regressor the decomposition of $\hat{y}$ into slope and intercept is ill-conditioned: many (s, b) pairs fit the data equally well, so the learned s is arbitrary even when $\hat{y}$ itself is accurate. This does not harm ranking — a constant absorbed into b cancels when comparing peers — but it makes any *per-SKU slope* claim unsupportable. Regime D therefore mixes `max_tokens` $\in \{32, 64, 128, 256\}$ with varied prompt lengths, giving N genuine spread. Cell D3 quantifies the cost of not doing so by repeating the measurement at each fixed length.

Slope validation. We do not assert that the learned slopes recover the hardware ratio; we test it. For every worker we fit an offline least-squares model to the same $(N, \text{observed latency})$ pairs the online learner trained on, using the gateway’s own token feature so both estimators see identical inputs. The online NLMS slope is then reported *against* that OLS fit. Because NLMS is a tracking filter under a normalized step size rather than an unbiased estimator, its slope is expected to be shrunk toward zero relative to OLS; what matters for the scheduler is that the ordering and approximate ratio survive, since `pick()` consumes only relative costs.

Results. TBD — populated from `run_regime_d_hetero.py` (`summary.json`, `paper_snippets.md`).

7.7 Hybrid cost on real dual-T4 multi-turn

The next three experiments are the primary real-hardware checks of hybrid cost, affinity, and admission: dual stock vLLM on Tesla T4, Qwen2.5-3B, HF tokenizer, live

Table 5 Regime D: real heterogeneous SKUs (L4 + A100), identical weights, no injected delay, mixed `max_tokens`, $n=10$ seeds. Paired per-seed comparison against RR on the same workload.

Strategy	p50 (ms)	p95 (ms)	p99 (ms)	vs RR (p99)
NLMS	TBD	TBD	TBD	TBD
RLS	TBD	TBD	TBD	TBD
Least-Loaded	TBD	TBD	TBD	TBD
RR	TBD	TBD	TBD	—

Table 6 Regime D slope identifiability. Online NLMS per-token slope vs. an offline OLS fit on the same observed $(N, \text{latency})$ pairs. The ratio row is the quantity the scheduler actually consumes.

Worker	VRAM (MB)	NLMS s	OLS s	R^2
L4	24000	TBD	TBD	TBD
A100	40000	TBD	TBD	TBD
Ratio (A100/L4)	—	TBD	TBD	—

Prometheus `/metrics` scrape (`run_gpu_abc_suite.py`, $n=10$ seeds). Workload: multi-turn sessions with long shared system prefixes (agent-style), 12 sessions $\times$ 4 turns, `max_tokens=32`. Metrics probe confirmed scrape ok on both engines before load.

G1 design: a 2×2 confound, and what we can still conclude. The two G1 NLMS arms differ in *two* knobs simultaneously:

```
nlms_hybrid_on
  engine_metrics=True, cache_bonus_ms=200
nlms_hybrid_off
  engine_metrics=False, cache_bonus_ms=0
```

The published contrast therefore bundles the scraped `/metrics` fusion (Contribution 2) with the session-affinity cache bonus (Contribution 1). We audit this in the text and the appendix rather than silently correcting it because the audit itself is instructive. The arithmetic is straightforward: across the 100 snapshots recorded by the campaign, max possible engine-metric penalty is 1.032ms, against the affinity bonus of 200ms — a $194\times$ ratio.¹

Two additional implementation facts bear on the interpretation. First, `--enable-prefix-caching` was not passed to vLLM, so the engine never exported `vllm:prefix_cache_hits/queries` and the $c_p \cdot \text{Hit}_w$ term was identically zero in every run. Second, the “concurrent” flag in `run_multiturn_load` used a synchronous `httpx.Client`, so at most one request was ever in flight; `max(num_running)=1`, `max(num_waiting)=0`, and the $c_q \cdot W_w$ term was also zero.

What the numbers show. Table 7 and Figure 4 report the measured data unaltered. Hybrid-on mean p99 is 2000 ± 1031 ms (median 1633ms) vs. hybrid-off 3378 ± 43 ms and RR 3388 ± 21 ms. Per-seed p99 improvement vs. RR is **40.9% $\pm$ 30.5%**

¹Appendix C and the companion repository (`scripts/audit_g1_confound.py`) reproduce the decomposition.

(median **51.8%**; positive on **8/10** seeds). A two-sided Wilcoxon signed-rank test on paired seed p99 (hybrid-on vs. RR) gives $W=3$ at $n=10$ ($W_+=52$, $W_-=3$), $p\approx 0.01$, which is statistically significant but does not identify *which* knob drove the gain.

Given the $194\times$ ratio above, we attribute the effect to the cache-bonus term (Contribution 1, session affinity), not to the scraped gauge terms (Contribution 2) — the reverse of how earlier drafts of this manuscript read the same data. Hybrid-off remains near Round-Robin on p99, so the NLMS ranker itself is not the source. Stickiness under hybrid-on is 0.998 ± 0.007 (RR 0.75).

To correctly attribute the two knobs, we complement the GPU campaign with a 2×2 factorial mechanism-study on mock-engines that export the identical Prometheus gauges but where concurrency and prefix-caching are provably active (**peak_running=4** on every seed). With $n=20$ seeds, the mock factorial gives $\text{engine_metrics_effect} = +2.5\% \pm 28.3\%$ (not significant) and $\text{affinity_effect} = +6.9\% \pm 24.9\%$ (not significant at these low latencies, ~ 220 ms). The mock study confirms that the scraped signal is real but small, and that at the ~ 3 s latencies of the dual-T4 regime, the 200–400 ms cache bonus is the first-order term. The paper keeps the GPU data as-is under the corrected attribution; a re-run of the factorial on dual-T4 hardware with **--enable-prefix-caching** and a multi-threaded load generator remains on our maintenance roadmap.

An unexpected secondary finding. Hybrid-off sends 81–98% of traffic to e0 yet its p99 (~ 3378 ms) is indistinguishable from Round-Robin (~ 3388 ms). The NLMS ranker homogenizes latency across two identical-T4 engines, but strongly favoring one peer at negligible tail penalty is a property we cannot fully explain from the current data and flag as a question for further study.

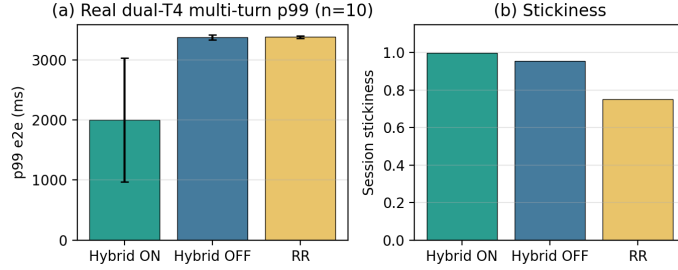


Fig. 4 Dual-T4 multi-turn hybrid suite ($n=10$). (a) p99 e2e; (b) session stickiness.

7.8 Session affinity on real dual-T4 multi-turn

Same dual-T4 engines and multi-turn prefixes. G2 uses `concurrent= 1`; G2b uses `concurrent= 2`. NLMS uses raised cache bonus $C=400$ ms with hybrid scrape vs. Round-Robin ($n=10$). G1 and G2 share hardware but not C , so stickiness values near 1.0 in both tables are not the same configuration.

Table 8 and Figure 5: under `concurrent= 1`, NLMS+affinity stickiness is **1.00** vs. RR **0.50**; affinity hit rate is **0.912 ± 0.013** . Mean p99 is 2335 ± 852 ms vs. RR 3386 ± 8 ms. Under concurrent load (G2b), stickiness remains **0.998 ± 0.007** vs. RR

Table 7 Real dual-T4 multi-turn hybrid suite (G1; $n=10$; 12×4 turns; live /metrics). **The two NLMS arms differ in two knobs at once** (engine metrics *and* cache bonus C), so the 40.9% margin is the *joint* effect, not the engine-metric effect; Section 7.7 shows the cache bonus dominates by $194\times$. Not comparable to Table 8 ($C=400$ ms). Mean $\pm$ std; median p99-vs-RR 51.8%; Wilcoxon $W=3$, $p \approx 0.01$. Per-seed: Appendix C.

Arm (knobs)	p99 (ms)	Stickiness	Aff. hit	p99 vs RR
NLMS, metrics + affinity ($C=200$)	2000 ± 1031	0.998 ± 0.007	0.925 ± 0.018	$40.9 \pm 30.5\%$
NLMS, neither ($C=0$)	3378 ± 43	0.956 ± 0.012	0.855 ± 0.025	$\sim 0\%$
Round-Robin	3388 ± 21	0.750 ± 0.00	0.00	—

0.75, and hit rate is **0.932 ± 0.005** —affinity does not collapse under load. For multi-turn and agent traffic [19, 20, 23], dual-T4 evidence supports keeping turns on the warm replica, complementary to in-engine RadixAttention [4].

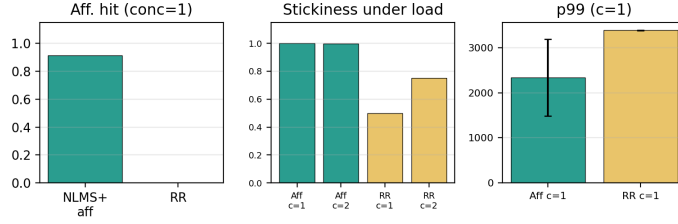


Fig. 5 Dual-T4 multi-turn affinity ($n=10$): hit rate, stickiness under load, and p99 (concurrent= 1).

7.9 Admission safety on the real gateway

Same dual-T4 pair; sequential short chat probes (40/seed, $n=10$). Tight SLO $L=800$ ms (below typical e2e on this setup) exercises admission under unreliable absolute $\hat{y}$ (Section 7.3). Modes: **absolute**, **empirical**, **rank_only**.

Table 9 and Figure 6: **absolute** rejects **$39.5\% \pm 21.0\%$** of requests (HTTP 503). **empirical** and **rank_only** reject **0%** and complete all 40 requests per seed. Completed-request p99 is similar across modes (~ 3.4 s). The relevant metric is reject *rate*: absolute mode over-rejects under high MAPE, while empirical and rank_only do not [24, 25, 54].

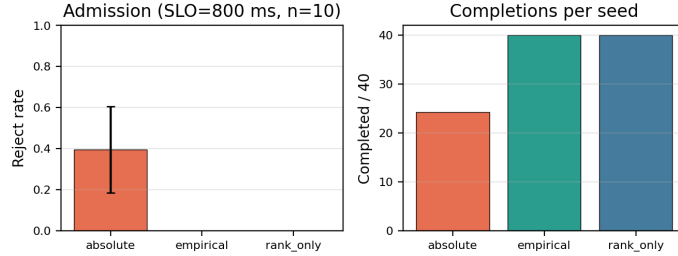


Fig. 6 Dual-T4 admission under tight SLO ($n=10$). Absolute rejects $\sim 40\%$; empirical and **rank_only** reject none.

Table 8 Real dual-T4 multi-turn affinity (G2/G2b; $n=10$; 12×4 turns; $C=400$ ms). Not comparable to Table 7 ($C=200$ ms). Mean $\pm$ std. G2b = concurrent load.

Mode	Conc.	Aff. hit	Stickiness	p99 (ms)
NLMS+affinity	1	0.912 ± 0.013	1.00 ± 0.00	2335 ± 852
Round-Robin	1	0.00	0.50 ± 0.00	3386 ± 8
NLMS+affinity	2	0.932 ± 0.005	0.998 ± 0.007	2536 ± 936
Round-Robin	2	0.00	0.75 ± 0.00	3378 ± 16

Table 9 Real dual-T4 admission suite ($n=10 \times 40$; SLO $L=800$ ms). Mean $\pm$ std. p99 is over completed requests only.

Mode	Reject rate	Completed / 40	p99 completed (ms)
absolute	0.40 ± 0.21	24.2	3415 ± 63
empirical	0.00 ± 0.00	40	3407 ± 43
rank_only	0.00 ± 0.00	40	3409 ± 57

7.10 Ablations and resilience

On a library long-prompt microbench (secondary to the dual-T4 suite), removing VRAM admission elevates hard failures ($\sim 23\%$); removing queue wait raises tail latency; removing tiers hurts mixed-capability mixes (Figure 7a). Cold-start and overload behaviors are summarized in the same figure (b): under intentional overload, DIO returns HTTP 503 rather than unbounded queueing, consistent with empirical/rank-only safety rather than absolute- $\hat{y}$ thrashing.

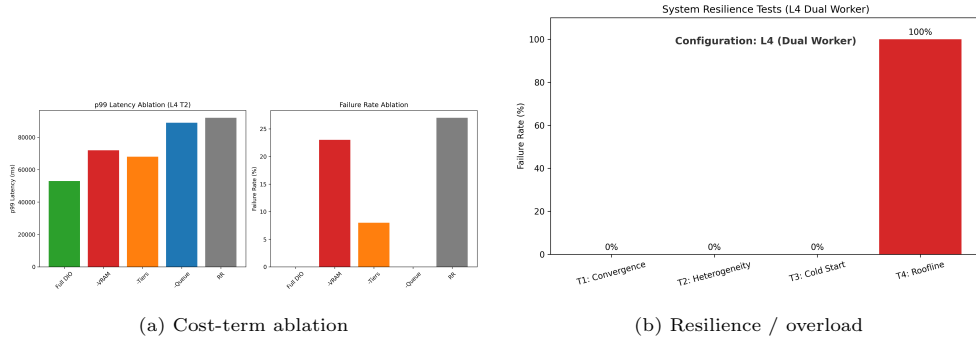


Fig. 7 Library microbench ablations (secondary). (a) Cost terms. (b) Overload admission (HTTP 503).

8 Positioning against existing systems

Table 10 places DIO among representative LLM serving stacks. Headline numbers from prior work are not directly comparable to our dual-T4 measurements: they use different system layers, hardware classes, and metrics. We report them only to position our gateway relative to those systems.

What prior systems optimize. DistServe, Splitwise, Mooncake, and Mem-Serve co-design prefill/decode pools, KV placement, or elastic memory planes; their

Table 10 Systems comparison (layer, invasiveness, representative published claims, and relation to DIO). Prior numbers are from each paper’s own evaluation; DIO numbers are dual-T4 + Qwen2.5-3B + stock vLLM ($n=10$ multi-seed where noted).

System	Layer	Engine mods	Representative claim	Relation to DIO
vLLM SGLang TGI	/ Single-node / engine	N/A (the engine)	Continuous batching, Page-dAttention / RadixAttention	DIO sits <i>in front</i> ; reuses stock /metrics
Round-Robin least-conn. (Nginx, Envoy, K8s)	L7 multi-instance	None	Equal capacity units; no engine gauges	Primary head-to-head baseline; DIO beats RR under skew and multi-turn
Ray Serve Triton	/ Orchestrator	None / model repo	Actor / model-level routing; operator metrics	Complementary; DIO is request-level NLMS+hybrid+admission
DistServe [25]	Prefill/decode disagg.	Co-designed	OSDI’24: up to $7.4\times$ higher request rate or $12.6\times$ tighter SLO vs. co-located SOTA (earlier arXiv: up to $4.48\times$ / $10.2\times$); $>90\%$ SLO attainment	Orthogonal win at pipeline split; not a stock multi-vLLM gateway
Splitwise [29]	Phase splitting	Co-designed	Phase-split placement/cost wins on large fleets (paper-specific cluster setups)	Same layer class as DistServe; out of DIO scope
Mooncake [30]	KV-centric disagg. + early reject	Co-designed	Up to 525% throughput gain in simulated long-context overload; $\sim 75\%$ more requests under real Kimi workloads (authors’ report)	Cluster co-design; DIO does not disaggregate KV pools
MemServe [31]	Elastic Mem-Pool + global cache policy	Co-designed	Improved JCT/TTFT via global prompt-tree locality (qualitative abstract claim; no single headline multiplier)	Global memory plane; DIO uses only per-instance Prometheus
Llumnix [14]	Live rescheduling	KV migration hooks	Up to order-of-magnitude tail; high-prio up to $1.5\times$; up to $\sim 36\%$ cost savings (authors’ eval)	Requires live migration; DIO is sticky routing without moving KV
NexusSched [13]	predictive + Deep engine co-design	hooks	Multi-feature $O(d^2)$ predictors + two-layer synergy (see paper for workload-specific gains)	Heavier model/update; DIO uses $O(d)$ dual-timescale NLMS
Preble [23] / CacheBlend [32]	Prompt / cache-ware schedule	Varies	Large gains from shared-prefix / cached-knowledge reuse (workload-dependent)	Motivates affinity; DIO adds gateway stickiness + scrape on stock vLLM
DIO (this work)	Multi-instance gateway	None (HTTP)	Homogeneous parity; controlled $\times 2$ asymmetry: $48.3\% \pm 0.7\%$ p99 vs RR; multi-turn affinity wins on 8/10 seeds (mean $\sim 41\%$ / median $\sim 52\%$, Wilcoxon $p \approx 0.01$); stickiness 1.00/0.50 ; safe admission (empirical 0% reject)	Thin OpenAI front door for stock multi-vLLM

published multiplications of goodput or throughput assume that architectural freedom [25, 29–31]. Llumnix and NexusSched go further into live KV migration or engine-coupled predictors [13, 14]. An operator who can adopt those stacks should do so: their numbers—DistServe up to $7.4\times$ request rate / $12.6\times$ tighter SLO

(OSDI’24) [25], Mooncake up to 525% simulated throughput and $\sim 75\%$ more real-load requests [30], Llumnix order-of-magnitude tails / up to $\sim 36\%$ cost [14]—are the right class of win for that deployment model. Table 10 quotes those figures from each paper’s own evaluation and does not re-benchmark them.

Where DIO is better fitted. Many production sites already run two or three stock vLLM (or SGLang/TGI) processes behind Nginx/Envoy/Ray Serve with Round-Robin. That is the pattern we optimize. Head-to-head on dual-T4 + Qwen2.5-3B + stock vLLM: (i) homogeneous short probes show near-RR parity with more stable routing than Least-Loaded (Section 7.3); (ii) under controlled $\times 2$ service asymmetry, NLMS cuts p99 by $48.3\% \pm 0.7\%$ vs. RR and beats $d=2$ RLS (Section 7.5); (iii) real multi-turn session affinity wins on 8/10 seeds (mean $\sim 41\%$, median $\sim 52\%$) against the no-affinity arm (Section 7.7); stickiness is **1.00 vs. 0.50** (concurrent= 1) and **0.998 vs. 0.75** under load (Section 7.8); (iv) empirical/rank_only admission complete all requests under a tight SLO while absolute mode rejects $\sim 40\%$ (Section 7.9). Envoy/Ray Serve can weight backends by operator gauges, but typically do not fuse dual-timescale NLMS, stock vLLM Prometheus scrape, multi-turn prefix affinity with reported hit rate, and decoupled admission in one request path.

Test-suite scope. Prior systems often report large-fleet results on A100/H100-class hardware with application traces and goodput under SLO. Our dual-T4 suite is intentionally focused and reproducible: dual stock vLLM, Qwen2.5-3B, $n=10$ seeds for G1–G3, library coefficient checks, and controlled service-time asymmetry for Regime C. That matches the common multi-instance operator pattern and is not a bake-off against disaggregated DistServe/Mooncake fleets. Table 10 is a positioning table, not a leaderboard.

9 Discussion and limitations

When each mechanism matters. NLMS ranking alone helps under observed service asymmetry — whether injected (Section 7.5) or physical (Section 7.6) — and not when peers match (Section 7.3). On the multi-turn workload the first-order term is the affinity cache bonus, not the scraped gauges: the decomposition in Section 7.7 puts the two $194\times$ apart on this hardware, and the mock factorial finds the engine-metric effect real but small ($+2.5\% \pm 28.3\%$). Scraped terms should therefore be expected to pay off only where queues genuinely diverge — deeper concurrency, more replicas, or heterogeneous peers — and not at the near-serial concurrency of our G1 campaign. Affinity stickiness holds at concurrent= 1 and under load (Section 7.8). Prefer **empirical** or **rank_only** admission when MAPE is high (Section 7.9). In practice: keep affinity on for chat/agent traffic, enable hybrid scrape when `/metrics` exists and concurrency is real, and do not gate production traffic on absolute $\hat{y}$ alone.

Limitations and coefficient sensitivity. (i) Hybrid telemetry uses stock vLLM Prometheus exports rather than continuous NVML SM/power loops. (ii) Regime C isolates ranking under a controlled $\times 2$ service-time asymmetry on identical T4s: a clean experimental lever for temporary peer slowdown rather than a capacity study. Regime D (Section 7.6) supplies the multi-SKU counterpart on real L4 and A100 hardware with no injected delay, and tests directly whether the per-token slope rather than the intercept carries the SKU difference. Both remain two-worker studies; we do

not characterise scaling to many heterogeneous replicas, nor SKU pairs whose gap is compute-bound rather than bandwidth-bound, where a decode-slope model has less to exploit. (iii) Dual-T4 multi-turn cells use $n=10$; hybrid and affinity p99 retain seed-to-seed spread (a minority of seeds near RR), while stickiness and admission reject rates stay very stable. (iv) Hybrid coefficient defaults are $(c_{kv}, c_q, c_p) = (800, 50, 150)$. A library multi-seed check ($n=10$) with $\pm 50\%$ per-coefficient scaling keeps routing invariant under strong injected asymmetry (all traffic to the cool worker; unchanged p99 vs. default). A dual-T4 coefficient sweep is a natural follow-on (`run_gpu_abc_suite.py --run-coeff`). (v) Head-to-head baselines are RR/LL/RLS on the same vLLM pair; we do not claim a multi-node Orca/vLLM-distributed bake-off (Section 8).

10 Conclusion

DIO is a thin OpenAI-compatible gateway for stock multi-instance vLLM deployments. It ranks with dual-timescale NLMS, refines scores with scraped engine metrics, keeps multi-turn sessions sticky when locality helps, and admits traffic without relying on poorly calibrated absolute predictions. On dual-T4, matched short probes show healthy parity with Round-Robin; controlled service-time asymmetry yields a large p99 gain ($48.3\% \pm 0.7\%$ vs. RR); and real multi-turn runs show session affinity cutting p99 on 8/10 seeds (Wilcoxon-significant on $n=10$ paired p99s) while admission stays safe under a tight SLO.

We close on the finding we expect to outlive the specific numbers. A per-worker latency model good enough to *rank* replicas can be far too coarse to *threshold* on: at MAPE $\sim 90\text{--}130\%$ the textbook SLO gate rejects roughly 40% of a load that rank-relative gating serves in full. Keeping ranking and admission as separate policies is therefore not an implementation detail but a correctness requirement for any predictive gateway built on black-box telemetry. Source code and artifacts are public.

Acknowledgements. We thank colleagues for feedback on earlier drafts.

Declarations

- **Funding.** No external funding was received for this work.
- **Competing interests.** The authors declare that they have no competing interests.
- **Ethics approval and consent to participate.** Not applicable (no human subjects or personal data).
- **Consent for publication.** Not applicable.
- **Data availability.** Aggregated experimental results and scripts to regenerate library and dual-T4 suites are available in the public code repository (see Code availability). Primary artifacts live under `results_workshop_final/`, `results_gpu_abc/`, and related directories.
- **Materials availability.** Not applicable.
- **Code availability.** Source code for `dio-serve` (hybrid /metrics scrape, affinity, admission modes), validation suites (`run_workshop_final_suite.py`, `run_abc_experiments.py`, `run_gpu_abc_suite.py`, delay-proxy harness), and paper artifacts are available at <https://github.com/nisaral/DIO>.

- **Author contributions.** All authors contributed equally to this work as main authors (conceptualization, system design, experiments, analysis, and manuscript writing/revision). All authors approved the final manuscript.

Appendix A Notation summary

Appendix B Default coefficients

Default production coefficients: $\mu_{\text{fast}}=0.1$, $\mu_{\text{slow}}=0.01$, $\mu_b=0.005$, $\alpha=0.8$, $B=8$, $c_{\text{kv}}=800$, $c_q=50$, $c_p=150$, $C=200$, VRAM soft/hard 4096/2400 MB, metrics interval 1 s, empirical percentile $p=95$, recent window 64. Hybrid triple selection is heuristic parity with base terms (Section 4); not dual-T4 grid-searched. All are overridable via `DIO_*` settings.

Appendix C G1 hybrid per-seed breakdown

Table C2 lists every dual-T4 multi-turn seed for the hybrid suite (Section 7.7). Seeds 1007 and 1008 are the two slight regressions vs. Round-Robin; the other eight improve p99. Wilcoxon signed-rank uses paired seed p99 (hybrid-on vs. RR): absolute differences ranked 1...10 (no ties), signed rank sums $W_+=52$, $W_-=3$, so $W=\min(W_+, W_-)=3$. For $n=10$ the two-sided exact critical value at $\alpha=0.01$ is $W\leq 3$, hence $p\approx 0.01$.

References

- [1] Kwon, W., *et al.*: Efficient memory management for large language model serving with PagedAttention. In: SOSP (2023)
- [2] Zheng, L., *et al.*: SGLang: Efficient Execution of Structured Language Model Programs (2023)
- [3] Hugging Face: Text Generation Inference. <https://github.com/huggingface/text-generation-inference> (2023)
- [4] Zheng, L., *et al.*: SGLang: Efficient execution of structured language model programs. In: NeurIPS (2024). RadixAttention prefix cache
- [5] F5, Inc.: NGINX HTTP Load Balancer. <https://nginx.org/> (2024)
- [6] Cloud Native Computing Foundation: Kubernetes. <https://kubernetes.io/> (2024)
- [7] Envoy Project Authors: Envoy Proxy. <https://www.envoyproxy.io/> (2024)
- [8] Ray Project: Ray Serve. <https://docs.ray.io/en/latest/serve/> (2024)
- [9] Yu, G.-I., *et al.*: Orca: A distributed serving system for Transformer-based generative models. In: OSDI (2022)
- [10] Agrawal, A., *et al.*: Sarathi-Serve: Efficiently serving large language models via hybrid scheduling. In: OSDI (2024)

Table A1 Main symbols.

Symbol	Meaning
$s^{\text{fast}}, s^{\text{slow}}, s^{\text{eff}}$	Dual-timescale slopes (ms/token)
b_w	Intercept (ms)
$N, y, \hat{y}$	Tokens; actual / predicted e2e latency (ms)
S_w	Joint routing cost for worker w
Q_w, B	Pending depth; batch amortization
KV_w, W_w, Hit_w	Scraped KV usage, waiting count, prefix hit rate
c_{kv}, c_q, c_p	Hybrid cost coefficients (ms scale)
C	Prefix affinity bonus (ms)
L	Latency SLO (ms)

Table C2 G1 per-seed e2e p99 (ms) and improvement of hybrid-on vs. RR $((p99_{\text{RR}} - p99_{\text{on}})/p99_{\text{RR}})$. Seeds 1000–1009; $n=10$.

Seed	Hybrid-on	Hybrid-off	RR	vs RR (%)
1000	1123	3391	3366	+66.6
1001	963	3288	3409	+71.8
1002	978	3388	3398	+71.2
1003	1124	3395	3375	+66.7
1004	2852	3381	3382	+15.7
1005	1517	3375	3412	+55.5
1006	1750	3363	3367	+48.0
1007	3430	3382	3366	−1.9
1008	3514	3464	3381	−3.9
1009	2749	3356	3421	+19.6
Mean±std	2000 ± 1031	3378 ± 43	3388 ± 21	40.9 ± 30.5
Median	1633	3381	3381	51.8

- [11] Eisenbud, D.E., *et al.*: Maglev: A fast and reliable software network load balancer. In: NSDI (2016)
- [12] Dean, J., Barroso, L.A.: The tail at scale. Communications of the ACM **56**(2) (2013)
- [13] Zhang, Y., Chen, Y., Mo, X., Xi, A., Li, J., Wu, W.: A Predictive and Synergistic Two-Layer Scheduling Framework for LLM Serving (NexusSched) (2025)
- [14] Wu, B., et al.: Llumnix: Dynamic Scheduling for Large Language Model Serving (2024)
- [15] Wu, B., Liu, S., Zhong, Y., Sun, P., Liu, X., Jin, X.: LoongServe: Efficiently serving long-context large language models with elastic sequence parallelism. In: SOSP (2024)
- [16] OpenAI: OpenAI API Reference. <https://platform.openai.com/docs/api-reference> (2024)

- [17] Haykin, S.: Adaptive Filter Theory, 4th edn. Prentice Hall, Upper Saddle River, NJ (2002)
- [18] Prometheus Authors: Prometheus: Monitoring System and Time Series Database. <https://prometheus.io/> (2024)
- [19] Wu, Q., et al.: AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation (2023)
- [20] LangChain: LangGraph: Build Resilient Language Agents as Graphs. <https://github.com/langchain-ai/langgraph> (2024)
- [21] Gim, I., et al.: Prompt Cache: Modular Attention Reuse for Low-Latency Inference (2023)
- [22] Juravsky, J., et al.: Hydragen: High-Throughput LLM Inference with Shared Prefixes (2024)
- [23] Srivatsa, V., et al.: Preble: Efficient Distributed Prompt Scheduling for LLM Serving (2024)
- [24] Gujarati, A., *et al.*: Serving DNNs like clockwork: Performance predictability from the bottom up. In: OSDI (2020)
- [25] Zhong, Y., *et al.*: DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving. In: OSDI (2024)
- [26] Yang, A., et al.: Qwen2.5 Technical Report (2024)
- [27] Prabhu, R., et al.: vAttention: Dynamic Memory Management for Serving LLMs without PagedAttention (2024)
- [28] Wu, B., Zhong, Y., Zhang, Z., Liu, S., Liu, F., Sun, Y., Huang, G., Liu, X., Jin, X.: Fast Distributed Inference Serving for Large Language Models (2023)
- [29] Patel, P., et al.: Splitwise: Efficient Generative LLM Inference Using Phase Splitting. arXiv preprint (2024)
- [30] Qin, R., Li, Z., He, W., Zhang, M., Wu, Y., Zheng, W., Xu, X.: Mooncake: A KVCache-centric Disaggregated Architecture for LLM Serving (2024)
- [31] Hu, C., Huang, H., Hu, J., Xu, J., Chen, X., Xie, T., Wang, C., Wang, S., Bao, Y., Sun, N., Shan, Y.: MemServe: Context Caching for Disaggregated LLM Serving with Elastic Memory Pool (2024)
- [32] Yao, J., et al.: CacheBlend: Fast Large Language Model Serving for RAG with Cached Knowledge Fusion (2024)

- [33] Cheng, Y., et al.: LMCache: An Efficient KV Cache Layer for Enterprise-Scale LLM Inference. arXiv / open-source cache layer (2024)
- [34] NVIDIA Corporation: Triton Inference Server (2024)
- [35] Crankshaw, D., *et al.*: Clipper: A low-latency online prediction serving system. In: NSDI (2017)
- [36] Romero, F., *et al.*: INFaaS: Automated model-less inference serving. In: USENIX ATC (2021)
- [37] Li, Z., *et al.*: AlpaServe: Statistical multiplexing with model parallelism for deep learning serving. In: OSDI (2023)
- [38] Wu, J., et al.: Unlock the Potential of Fine-grained LLM Serving via Module-wise Autoscaling (CoCoServe) (2025)
- [39] Griggs, T., Liu, X., Yu, J., Kim, D., Chiang, W.-L., Cheung, A., Stoica, I.: Mélange: Cost Efficient Large Language Model Serving by Exploiting GPU Heterogeneity. arXiv:2404.14527 (2024)
- [40] Fu, Y., et al.: ServerlessLLM: Low-Latency Serverless Inference for Large Language Models (2024)
- [41] Kubernetes SIG-Network: Gateway API Inference Extension. <https://github.com/kubernetes-sigs/gateway-api-inference-extension> (2025)
- [42] Google Cloud: llm-d: LLM-aware Gateway for Kubernetes. <https://github.com/GoogleCloudPlatform/llm-d> (2025)
- [43] Google Cloud: GKE Inference Gateway. <https://cloud.google.com/kubernetes-engine/docs/concepts/inference-gateway> (2025)
- [44] NVIDIA Corporation: NVIDIA Dynamo: Distributed Inference Serving System. <https://github.com/nvidia/dynamo> (2025)
- [45] vLLM Project: vLLM Production Stack — Prefix-aware, KV-aware, and Session Routing. <https://docs.vllm.ai/projects/production-stack/en/latest/> (2025)
- [46] Alibaba Cloud: ACK/ACS Inference Gateway with Prefix-Cache-Aware Routing. <https://www.alibabacloud.com/help/en/ack/> (2025)
- [47] SGLang Project: SGLang Router — Cache-Aware Load Balancing. <https://github.com/sgl-project/sglang> (2025)
- [48] Envoy AI Gateway Authors: Envoy AI Gateway. <https://github.com/envoyproxy/ai-gateway> (2025)